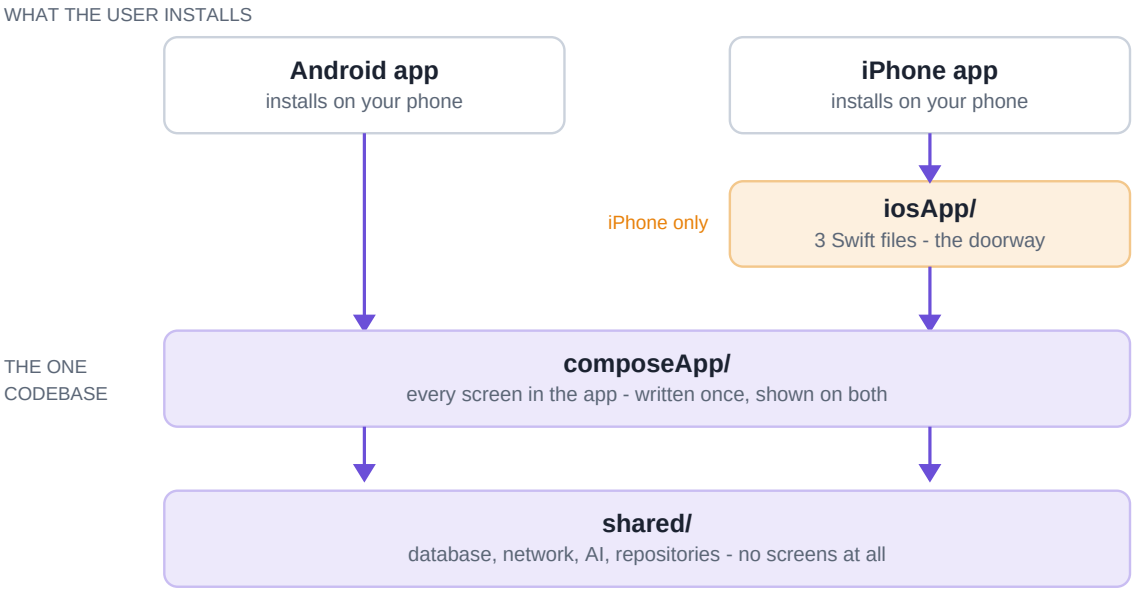
FixMoney KMP

How the project is put together - a picture-led map of the folders, what lives where, and why.

The one idea behind the whole thing

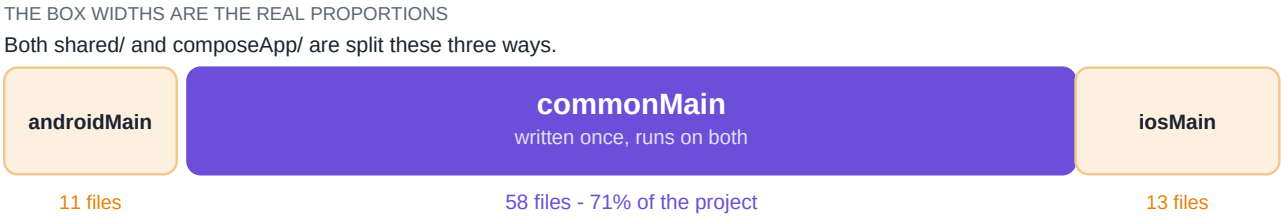
FixMoney is **one codebase that becomes two apps** - an Android app and an iPhone app. Most of it is written once and used by both. Only the few parts that genuinely differ between the two phones are written twice.



A short way to remember it: **shared** knows things, **composeApp** shows things, and **iosApp** is just the doorway iOS insists on.

Three source sets, in every folder

Both `shared/` and `composeApp/` split the same three ways. In the picture below the **widths are the real file counts** - that is how lopsided it is in the good direction.



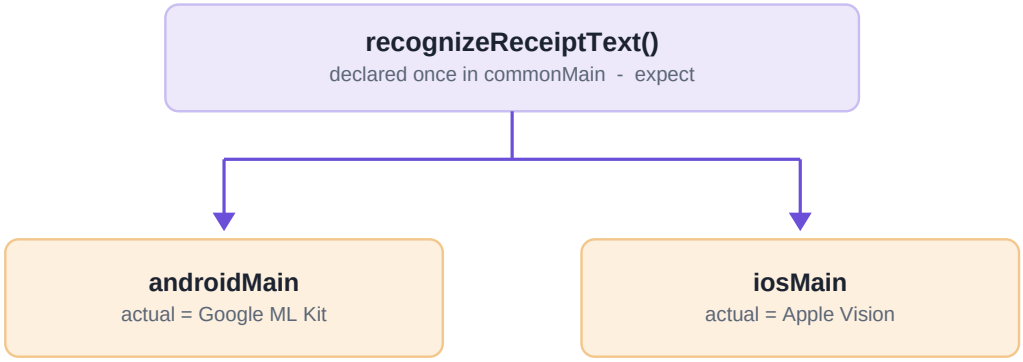
Android-only code and iPhone-only code are the narrow strips at the edges. Everything in the middle is written one time.

Source set	Meaning
`commonMain/`	Written once, compiled into **both** apps. Most code lives here.
`androidMain/`	Android only. Can use any Android API.
`iosMain/`	iPhone only. Can use any iOS API - UIKit, Vision, and so on.

How 'write once, differ where needed' works

Common code states what it needs with `expect`. Each platform supplies its own `actual`. Reading the text off a receipt photo is the clearest example - there is no single library that does it on both phones, so each side brings its own.

ONE PROMISE, TWO ANSWERS



Only this one step differs. Everything around it is shared.

```
// commonMain - the promise
expect suspend fun recognizeReceiptText(bytes: ByteArray): String

// androidMain - kept by Google ML Kit
actual suspend fun recognizeReceiptText(...) = mlKitRecognizer.process(...)

// iosMain - kept by Apple's Vision framework
actual suspend fun recognizeReceiptText(...) = VNRecognizeTextRequest(...)
```

Everything above that line - judging the photo too blurry, sending the text to the AI, matching a category, saving the row - is written one time.

What sits inside shared/

37 files. No screens, no colours, no buttons. If both apps vanished tomorrow, this folder would still make sense on its own.

shared/ - 37 FILES, GROUPED BY JOB

Stores things	Fetches things	Thinks	Hands things over
db/ - Room database	data/ - categories from Firebase	ai/ - chat, voice, receipt parsing	expense/ - repositories
onboarding/ - profile, settings	platform/ - HTTP, logging, date	receipt/ - scan pipeline	model/ - Category, Currency
		voice/ - quick-add flow	di/ - Koin wiring
		home/ - dashboard numbers	

What sits inside composeApp/

42 files. Every screen, written once, shown on both phones.

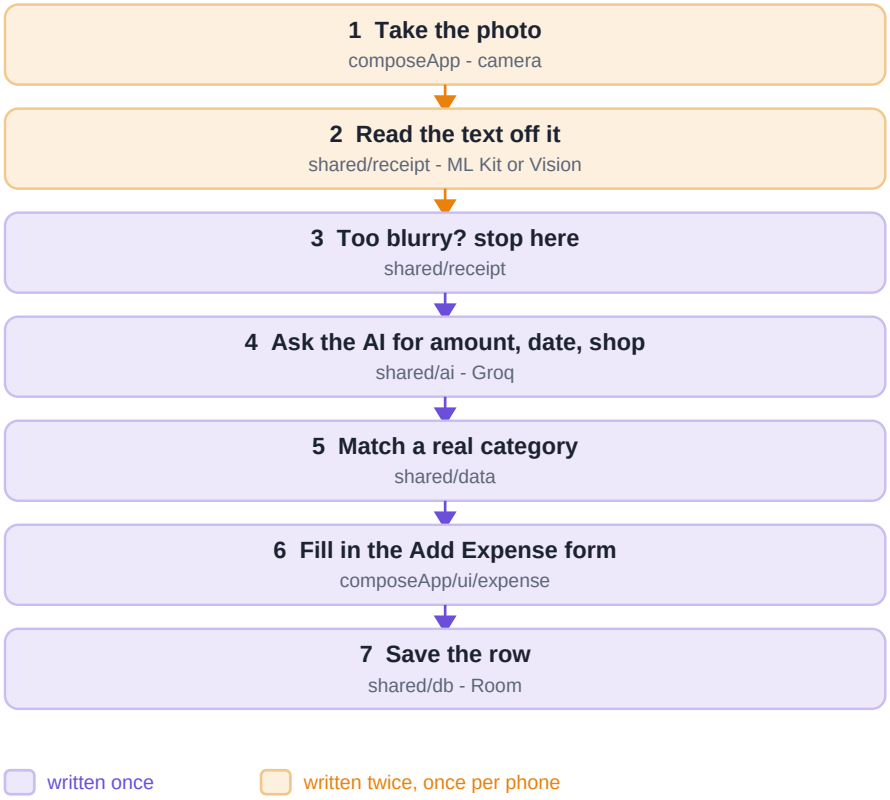
Folder	Screens inside
`ui/onboarding/`	First run: welcome, three feature pages, name, budget, currency, all set.
`ui/home/`	The five-tab shell, the Home dashboard, the spending donut.
`ui/expense/`	Add Expense, and the Expenses list with search and filters.
`ui/budget/`	Budget per category, and the Set Budget dialog.
`ui/analytics/`	Six months of spending as bars, plus top categories.
`ui/scanner/`	The receipt scanner: capture, review, scan.
`ui/chat/`	Fixoo, the AI chat assistant.
`ui/settings/`	Settings, SIP calculator, export dialog.
`ui/components/`	Pieces reused everywhere - buttons, the month and year picker.
`ui/` root	App.kt (navigation), Theme.kt (59 colour tokens), fonts, sizing.

Following one feature all the way through

Scan Receipt is the best example, because it touches every layer at once.

SCAN RECEIPT, END TO END

Five of the seven steps are written one time.



Two steps out of seven are written twice. The other five are written once - and those five are where all the actual thinking happens.

The only places the two apps part ways

Feature	Android	iPhone
Voice quick-add	A floating bubble over every other app.	**Not possible.** iOS forbids drawing over other apps. A Siri shortcut does the same job instead.
Receipt text	Google ML Kit.	Apple's Vision framework.
Receipt camera	Built-in camera with an alignment frame that crops the shot.	The system camera, with its own crop step.
System bars	Status and navigation bars tinted to match the theme.	Handled by iOS itself.

iosApp/ - the doorway

Only three Swift files. Xcode opens this folder, never the project root - Xcode cannot open a Gradle project.

File	What it does
`iOSApp.swift`	The app entry point.
`ContentView.swift`	Hands the whole window over to the shared Compose screens.
`AddExpenseIntent.swift`	The Siri shortcut - 'Hey Siri, add an expense to FixMoney'. Runs with the app closed and calls the same shared code the Android bubble does.

Running it

```
export JAVA_HOME=.../ms-17.0.15/Contents/Home
./gradlew checkAll      # builds Android and compiles both modules for iOS
```

Open the **repo root** in Android Studio for all Kotlin work. Open **iosApp/** in Xcode for anything Swift.

Two rules worth remembering

1. The old app is reference material, not source. The original Fragments-and-XML FixMoney still ships on its own and is never touched. Screens here were rebuilt by looking at what the old app does, not by copying files.

2. Put code in the lowest folder that can hold it. If it does not need an Android or iOS API, it belongs in commonMain - then it only has to be written, fixed and tested once.